Servis Bağımlılıklarının Yazılım Performansı Üzerindeki Etkisi

Özet: .NET 9.0 ve Gateway mimarisi üzerinden mikroservis dayanıklılık testleri.

AHSEN ASLAN

Problem Tanımı ve Araştırma Sorusu

Bu çalışma, dağıtık mimarilerde servis bağımlılıklarının sistem performansı üzerindeki etkisini deneysel bir yaklaşımla incelemektedir. Araştırma kapsamında, .NET Core tabanlı bir API Gateway üzerinden farklı ağ gecikmeleri, işlemci yükleri ve hata koşulları test edilmiştir. Apache Benchmark ve JMeter araçlarıyla yapılan ölçümler; servis bağımlılıklarının yanıt sürelerini artırırken saniyedeki istek kapasitesini (throughput) önemli ölçüde düşürdüğünü saptamıştır. Elde edilen bulgular, dağıtık sistemlerde performans kayıplarının önlenmesinde devre kesici (circuit breaker) ve optimize edilmiş zaman aşımı (timeout) yapılandırmalarının kritik rolünü ortaya koymaktadır.kesici (circuit breaker) ve optimize edilmiş zaman aşımı (timeout) konfigürasyonlarının gerekliliği deneysel verilerle kanıtlanmıştır.

Research Question: "How do stochastic dependency failures affect architectural resilience in gateway-centric systems?"

Proje ve Tez Durumu

- Proje Tamamlanma: %100 (Tüm senaryolar ve random hata motoru tamamlandı).
- Tez Yazım Durumu: %90 içeriklerin düzenlenmesi yazımların kontrol edilmesi gibi ufak detaylar kaldı

Teknik Altyapı

Proje, yüksek performanslı ve modern bir dağıtık sistem simülasyonu olarak kurgulandı:

- Framework: .NET 9.0 (En güncel sürüm, performans odaklı iyileştirmeler için).
- Web Sunucusu: Kestrel (Yüksek verimli, asenkron işlem yeteneği için).
- Mimari: Gateway Pattern (İstemci -> Gateway -> External API).
- Test Araçları: * JMeter: Sistem seviyesinde yük testi ve metrik toplama.
    - BenchmarkDotNet: Kod seviyesinde mikro-maliyet analizi.
    - Apache Benchmark (ab): Hızlı RPS doğrulamaları.

Proje ve Tez Durumu

Koşulan Senaryolar (TC-01 - TC-08)

- Deneyler, sistemin sınırlarını zorlamak için 8 farklı aşamada gerçekleştirildi:
- TC-01 (Baseline): Hiçbir bağımlılık olmadan sistemin saf hızı.
- TC-02 (Level 1): Tek bir harici API bağımlılığının maliyeti.
- TC-03 (Hybrid): Hem CPU (Hashing) hem I/O yükünün birleşimi.
- TC-04 (Resilience): Kontrollü gecikmeler (Latency injection) altında sistem cevabı.
- TC-05 (Connection Failure): Dış servisin tamamen kapalı olma durumu (Hard Fail).
- TC-06 (Payload): 10MB gibi büyük veri paketlerinin transfer maliyeti.
- TC-07 (Pool Exhaustion): İşletim sistemi seviyesindeki soket kısıtlarının (MaxConnections=2) etkileri.
- TC-08 (Stochastic Error): %20-30 ihtimalle rastgele fırlatılan hatalar (Chaos Engineering).

Deneyler sonucunda 3 tane mühendislik bulgusuna ulaştık:

1. En Büyük Risk: "Zaman Aşımı" (TC-05)

Bulgu: Dış servis kapandığında sistem 9676 ms (yaklaşık 10 saniye) boyunca asılı kalıyor.

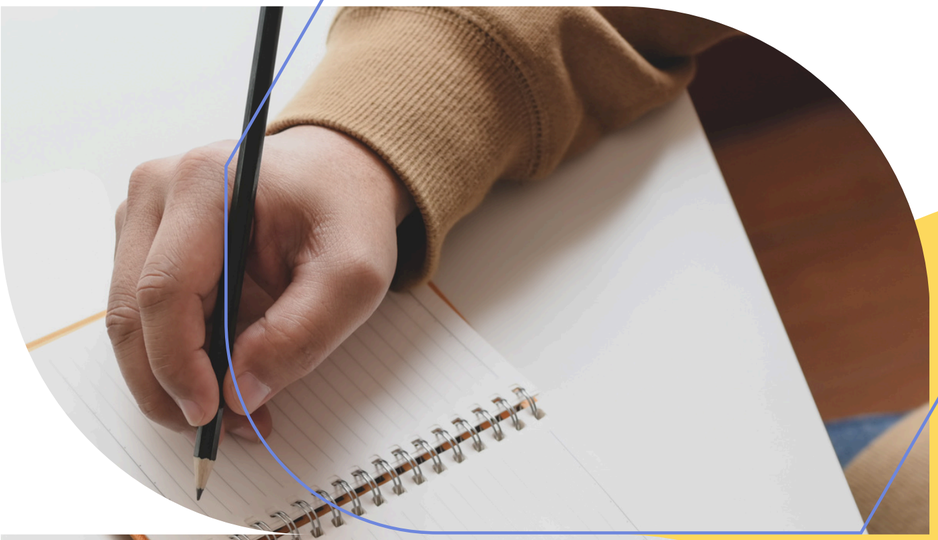
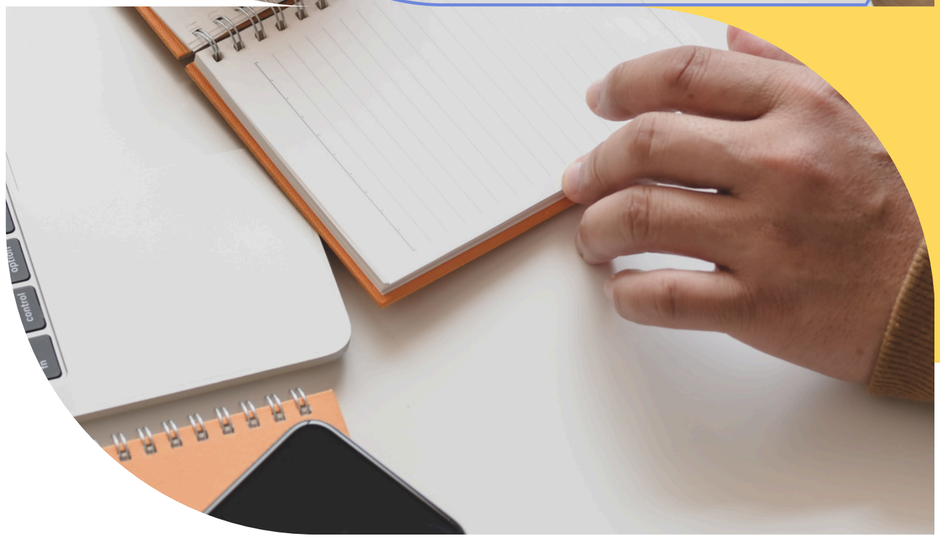
Anlamı: Gateway, dış servisin öldüğünü anlamak için çok bekliyor. Bu sürede gelen tüm istekler thread'leri tüketiyor. Bu, sistemin tamamen çökmesine (Cascading Failure) davetiye çıkarmaktır.

2. En Büyük Kurtarıcı: "Fail-Fast" (TC-08)

Bulgu: Sisteme rastgele hatalar enjekte ettiğimizde, hatalı isteklerin 2 ms içinde sonlandığını gördük.

Anlamı: Mimarimiz, hata aldığında kullanıcıyı boşuna bekletmiyor. Kaynakları (CPU/Thread) hemen serbest bırakıyor. Bilgin Hoca'nın istediği "random" senaryoda sistemin bu refleksinin ne kadar hayati olduğunu kanıtladık.

3. Yazılım vs. Altyapı (TC-07)

Bulgu: Kodumuz saniyede binlerce isteği işleyebilecek kapasitedeyken, bağlantı havuzunu (Socket Pool) kısıtladığımızda hızın 6 RPS seviyesine kadar düştüğünü gördük.

Anlamı: Yazılım ne kadar "resilient" olursa olsun, altyapı konfigürasyonu (Connection Pooling) doğru yapılmazsa mimari çöküyor.